

Agent Context Awareness: Current State & Recommendations

1. Current State: What Does a Spawned Agent See?

When the coordinator spawns an agent (via `spawn_agent_inner` in `src/commands/spawn.rs`), the prompt is assembled from the `TemplateVars` struct (`src/service/executor.rs:17-27`) with these slots:

Template Variable	Source	Content
{{skills_preamble}}	.claude/skills/using-superpowers/SKILL.md if present	Skill invocation discipline (wrapped in <EXTREMELY_IMPORTANT> tags)
{{task_identity}}	Agency: agent hash → role + motivation + skills	Role name, description, skills content, desired outcome, acceptable/unacceptable trade-offs
{{task_id}}	Task field	e.g. research-agent-context
{{task_title}}	Task field	e.g. Research agent context
{{task_description}}	Task field	Free-text description from <code>wg add -d</code>
{{task_context}}	<code>build_task_context()</code> in <code>spawn.rs</code>	Artifacts + last 5 log entries from each direct dependency
{{task_loop_info}}	Cycle config on task	Iteration number, max iterations, --converged instructions
{{working_dir}}	Parent of <code>.workgraph/</code> dir	Sets the <code>cwd</code> for the agent process
{{model}}	Model hierarchy resolution	Not directly visible to the agent in the prompt

The prompt template itself (lines 372-444 of `executor.rs`) then wraps these variables in a structured format with sections: Task Assignment header, Your Task, Context from Dependencies, Required Workflow (log/artifact/done/fail commands), Graph Patterns reference, Reusable Workflow Functions, and a critical warning about using `wg` CLI instead of built-in tools.

What's Missing

The agent does **not** receive:

- **wg context output** for its task (the structured inputs/artifacts view)
- **Upstream task descriptions** (only their artifacts and log tails)
- **Downstream task awareness** (what tasks depend on this one)
- **Graph topology** (no sense of where it sits in the DAG)
- **Project-level purpose/goal** (no top-level description of what the workgraph is for)
- **wg status / wg list summary** (no sense of overall project progress)

- **CLAUDE.md content** (this comes via Claude Code's own mechanism, not the prompt template)
- **Task tags** (the agent doesn't know its own task's tags)
- **Task skills list** (the role skills are injected, but the task-level `--skill` values are not explicitly shown)

2. Task-Local Context

Upstream task outputs/artifacts?

Partially. `build_task_context()` (spawn.rs:89-130) iterates `task.after` and collects:

- Artifact file paths from each dependency
- Last 5 log entries (reversed, then re-reversed) from completed dependencies

This is the *only* upstream context. Agents see "From dep-1: artifacts: output.txt, data.json" and "From dep-1 logs: ". They do **not** see the upstream task's description, title, or full output content.

wg context output?

No. The `wg context` command (context.rs) provides a richer view — declared inputs, available/missing input status, dependency task titles. None of this structured view is injected into the prompt. The agent could run `wg context` itself but doesn't know to.

Role and motivation descriptions?

Yes, if agency is configured. `resolve_identity()` (executor.rs:94-147) looks up the agent's hash, finds the agent entity, resolves role + motivation, and renders via `render_identity_prompt()`. This produces a well-structured "Agent Identity" section with role name/description, skills content, desired outcome, acceptable trade-offs, and non-negotiable constraints.

Skills it was assigned for?

Role-level skills: yes. Resolved from the role's skill references and rendered as mark-down sections with their content. **Task-level skills: no.** The task's `skills` field (from `wg add --skill x`) is not directly shown in the prompt, though it's used by the assigner to pick the right agent.

3. Graph Awareness

Currently agents see **only their task** plus dependency artifacts/logs. Here's an analysis of broader options:

Option A: Just their task (current)

- **Pro:** Minimal token cost, no confusion, no prompt injection risk from peer tasks
- **Con:** Agent can't make informed decisions about how its work fits the bigger picture. Can't avoid duplicating work done by siblings. Doesn't know what downstream consumers need.

Option B: Task + immediate neighbors (upstream/downstream)

- **Pro:** Agent knows what it's receiving and who it's producing for. Can tailor output format to downstream consumers. Low additional token cost (200-500 tokens).
- **Con:** Slight prompt injection risk if peer task descriptions contain adversarial content.

- **Implementation:** Add downstream task IDs + titles to context. Already have upstream artifacts.

Option C: N-hop neighborhood subgraph summary

- **Pro:** Better situational awareness for complex graphs.
- **Con:** Rapidly growing token cost. Diminishing returns past 1-hop. Complexity in implementation.

Option D: Full graph summary (wg status output)

- **Pro:** Maximum awareness.
- **Con:** Token-expensive for large graphs. Most of the information is irrelevant. Significant prompt injection surface.

Option E: Project-level purpose/goal

- **Pro:** Very cheap (one sentence). Grounds the agent's decisions in the "why." Currently, agents have no idea what the overall project is trying to accomplish.
- **Con:** Requires the user to set a project description (e.g., `wg config --project-description "..."`).

Assessment: Option B + E is the sweet spot. Showing immediate downstream tasks (ID + title only) and a one-line project purpose gives agents just enough context without token bloat or injection risk.

4. System Awareness

The current prompt tells agents to "use wg CLI" and lists specific commands (log, artifact, done, fail, add), but doesn't explain:

- What workgraph *is* conceptually (a directed graph of tasks with dependencies)
- What the coordinator does (polls for ready tasks, spawns agents, monitors health)
- How the agency system works (roles, motivations, agent assignment)
- What cycles/loops mean
- How trace functions work

Should it explain more?

Mostly no. The current approach is correct — agents are operators, not architects. They need to know *which commands to run*, not the internal design.

However, two pieces of system knowledge would help:

1. **"Your outputs flow downstream."** Agents should understand that their artifacts and logs are consumed by dependent tasks. This encourages better logging and artifact recording. Currently agents log perfunctorily because they don't understand the downstream impact.
2. **"You can discover context with `wg context <task-id>` and `wg show <task-id>`."** The prompt lists these in quickstart but not in the agent prompt template itself. Agents that need more information about their dependencies don't know how to get it.

Explaining coordinator internals, cycles, or trace functions would add tokens without improving task execution.

5. Recommendations

Ordered by impact/cost ratio (highest first):

R1: Add downstream task awareness to context (High impact, Low cost)

Add downstream task IDs and titles to `build_task_context()`. Currently:

From dep-1: artifacts: output.txt

From dep-1 logs: ...

Add:

Downstream consumers of your work:

- review-code: "Review implementation"
- integrate-results: "Merge all outputs"

This is 3 lines of code in `build_task_context()` — iterate `task.before` and print ID + title. Agents will produce better-targeted artifacts and documentation.

R2: Inject wg context hint into prompt (High impact, Trivial cost)

Add one line to the "Important" section of the prompt template:

- Run ``wg context {{task_id}}`` to see what your dependencies produced
- Run ``wg show <task-id>`` to inspect any task in the graph

Cost: 20 tokens. Helps agents self-serve when they need more information.

R3: Add project description to prompt (Medium impact, Low cost)

Add a `project_description` field to `config.toml` and a `{{project_description}}` template variable. Even a one-liner like "Building a task orchestration system in Rust" helps agents make better judgment calls about scope and quality.

R4: Include task tags and skills in prompt (Medium impact, Trivial

cost) The task's tags and skills fields are already in the Task struct but not rendered in the prompt. Adding:

- `**Tags:**` evaluation, code-review
- `**Skills:**` rust, analysis

helps agents understand what kind of task this is and what's expected.

R5: Show upstream task titles (not just artifacts) (Low-Medium

impact, Trivial cost) Currently `build_task_context()` shows "From dep-1: artifacts: file.txt" but not "From dep-1 (Write implementation code): artifacts: file.txt". Adding the title gives agents context about *what* produced the artifacts they're receiving. One-line change in the format string.

R6: Let agents opt into more context via `wg show --neighbors` (Low

impact, Medium cost) A new flag `wg show <task-id> --neighbors` that prints the task plus its 1-hop neighborhood (upstream + downstream with status). This is a self-serve escape hatch for agents that need more situational awareness. Not in the default prompt, but available on demand.

Not Recommended

- **Full graph summary in prompt:** Too many tokens, too much noise, prompt injection risk
- **Explaining coordinator internals:** Agents don't need to know how they were spawned

- **Explaining cycles/trace functions:** Only relevant for cycle header tasks (already handled by `{{task_loop_info}}`)
- **Injecting CLAUDE.md content:** Already handled by Claude Code's own mechanism; duplicating it would waste tokens

Summary Table

#	Change	Tokens Added	Files Changed	Impact
R1	Downstream task IDs in context	50-100	spawn.rs	High
R2	<code>wg context</code> hint in prompt	20	executor.rs	High
R3	Project description	20-50	executor.rs, config.rs	Medium
R4	Tags + skills in prompt	20-40	executor.rs	Medium
R5	Upstream task titles	10-20	spawn.rs	Low-Medium
R6	<code>wg show --neighbors</code>	0 (on demand)	show.rs	Low